

CS222 Assignment 4: เรียนรู้ Pthread Programming Part 1

Out: 3 เมษายน 69

250 คะแนน

Due: 7 เมษายน 69

Assignment นี้เป็นงานเดี่ยว Assignment มีวัตถุประสงค์ให้ เรียนรู้การเขียนโปรแกรม pthread programming เบื้องต้น

นโยบายเกี่ยวกับการใช้ AI:

1. Snipped นโยบายเปลี่ยนไปตามรายวิชาและเป้าหมายการฝึกฝน นศ

นโยบายเกี่ยวกับกระบวนการเรียนรู้:

1. Snipped นโยบายเปลี่ยนไปตามรายวิชาและเป้าหมายการฝึกฝน นศ

การส่งงาน:

1. จงเขียนคำตอบใส่ในรายงานในรูปแบบไฟล์ pdf พร้อมทั้งเขียนชื่อและรหัส นศ ทุกคนในกลุ่มให้ชัดเจน และส่งงานในระบบ MS Team กำหนดให้ ทุก Assignment ต้องมี Assignment Header ที่ระบุข้อมูลต่อไปนี้ที่ต้นรายงานอย่างชัดเจน ได้แก่
Assignment: Assignment ตามด้วยหมายเลข Assignment
Students: รายชื่อ นศ และ รหัส นศ ของผู้ร่วมงานทุกคน
หมายเหตุ: ต้องระบุรายชื่อที่ต้นรายงานเท่านั้นห้ามใส่ชื่อท้ายรายงานหรือที่อื่น
2. ในกรณีที่มีการคัดลอกงานหรือส่วนหนึ่งของงาน ทั้งผู้คัดลอกและผู้ให้ลอก จะได้รับ 0 คะแนน และจะถูกเตือน ถ้ายังมีการกระทำอีกจะมีการดำเนินการตามกฎหมายของมหาวิทยาลัย
3. ในกรณีส่งงาน late จะมีการตัดคะแนนวันละ 10 เปอร์เซนต์
4. เมื่อ upload ไฟล์ ครั้งแรกแล้ว นศ สามารถ upload ไฟล์เวอร์ชันใหม่ได้ โดยจะถือว่าเวลาในการ upload ครั้งล่าสุดเป็นเวลาส่ง (ถ้า upload ล่าสุดภายใน due date จะไม่ถูกหักคะแนน แต่ถ้าหลังจากนั้นจะถูกหักคะแนน)

การส่งโปรแกรม:

1. สำหรับคำถามที่มีการเขียนโปรแกรม ให้แนบ source code พร้อมทั้ง comments เขียนอธิบาย code สั้นๆด้วยคำพูดของ นศ เองในรายงาน และ capture ผลลัพธ์ของการรันโปรแกรมใส่ในรายงาน
2. เมื่อโปรแกรมเริ่มต้นประมวลผลให้ พิมพ์ชื่อภาษาอังกฤษ และ รหัส นศ ของ นศ ออกสู่หน้าจอ ของทุกโปรแกรม
3. ให้ นำ source code ของทุกโปรแกรมใส่ใน directory และสร้าง tar file ของ directory นั้นและ upload บน MS team พร้อมกับรายงาน

ข้อ 1: จาก Textbook “Operating System Concepts Edition 10” ในส่วนของ Programming Problem ข้อ 3.21 ที่มีเนื้อหาดังภาพที่ 1 และนิยามเกี่ยวกับ Fibonacci Number จาก https://en.wikipedia.org/wiki/Fibonacci_sequence ในภาพที่ 2

3.21 The Collatz conjecture concerns what happens when we take any positive integer n and apply the following algorithm:

$$n = \begin{cases} n/2, & \text{if } n \text{ is even} \\ 3 \times n + 1, & \text{if } n \text{ is odd} \end{cases}$$

The conjecture states that when this algorithm is continually applied, all positive integers will eventually reach 1. For example, if $n = 35$, the sequence is

35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1

Write a C program using the `fork()` system call that generates this sequence in the child process. The starting number will be provided from the command line. For example, if 8 is passed as a parameter on the command line, the child process will output 8, 4, 2, 1. Because the parent and child processes have their own copies of the data, it will be necessary for the child to output the sequence. Have the parent invoke the `wait()` call to wait for the child process to complete before exiting the program. Perform necessary error checking to ensure that a positive integer is passed on the command line.

ภาพ 1

The Fibonacci numbers may be defined by the [recurrence relation](#)^[7]

$$F_0 = 0, \quad F_1 = 1,$$

and

$$F_n = F_{n-1} + F_{n-2}$$

for $n > 1$.

ภาพ 2

จงตอบคำถามต่อไปนี้

- ศึกษาด้วยตนเองและตอบอย่างสั้นๆในรายงาน ว่า Collatz conjecture คืออะไร และ Fibonacci Number คืออะไร โดยต้องระบุแหล่งที่มาของข้อมูลอย่างชัดเจน
- จงเขียนและรันโปรแกรมภาษา C โดยมีข้อกำหนดต่อไปนี้
 - รับค่า Positive Integer N เป็น argument จาก Command line
 - กำหนดให้ N เป็น local variable ใน main function

- c. ถ้ารหัส นศ ของ นศ เป็นเลขคู่ ให้สร้าง starting routine ชื่อ `clz` โดยผ่านค่า `N` เป็น argument เพื่อพิมพ์ Collatz conjecture สำหรับค่า N ออกสู่หน้าจอ หาผลรวมของค่าตัวเลขที่พิมพ์ทุกค่า แล้วส่งค่าผลรวมนั้นกลับมาให้เทรด `main` โดยใช้คำสั่ง `pthread_exit()`
- d. ถ้ารหัส นศ ของ นศ เป็นเลขคี่ ให้สร้าง starting routine ชื่อ `fib` โดยผ่านค่า `N` เป็น argument เพื่อพิมพ์ Fibonacci number จำนวน N ค่า ออกสู่หน้าจอ หาผลรวมของค่าตัวเลขที่พิมพ์ทุกค่า แล้วส่งค่าผลรวมนั้นกลับมาให้เทรด `main` โดยใช้คำสั่ง `pthread_exit()`
- e. ให้ `main` ใช้คำสั่ง `pthread_join()` เพื่อรอให้เทรดที่สร้างจบการทำงาน
- f. ให้ `main` พิมพ์ค่าต่อไปนี้ออกสู่หน้าจอ
 - i. ค่ารหัส นศ ของ นศ (ถ้าไม่พิมพ์จะไม่ได้รับคะแนนในข้อนี้) และ
 - ii. ค่าผลรวมที่ได้รับเป็นค่าสถานะจากเทรดที่จบการทำงานสู่หน้าจอ

ข้อ 2: จงเขียนโปรแกรมเพื่อสร้าง thread (ด้วยคำสั่ง `pthread_create()`) เพื่อเรียก function เพื่อพิมพ์ “Hello World” ออกสู่หน้าจอจำนวน 50 threads และให้ `main thread` รอการจบการทำงานของทั้ง 50 threads นั้นด้วย `pthread_join()`

Hints:

1. ต้องสร้าง array เพื่อเก็บค่า Thread ID
2. ศึกษา pthread Tutorial ได้ที่

<https://hpc-tutorials.llnl.gov/posix/>

https://hpc-tutorials.llnl.gov/posix/joining_and_detaching/

<https://www.cs.cmu.edu/afs/cs/academic/class/15492-f07/www/pthreads.html>

ข้อ 3: จงตอบว่าเพราะเหตุใด ถ้าเราสร้างเทรดขึ้น 2 เเทรดให้ประมวลผลไปพร้อม ๆ กัน และทั้ง 2 เเทรดเข้าถึงตัวแปร สมมุติว่าชื่อ `int count`; เป็น global variable ที่มีค่าเริ่มต้นคือ 0 และกำหนดให้เทรดทั้งสองรันคำสั่ง `count++` ไปพร้อม ๆ กัน โดยไม่ใช้ mutual exclusion synchronization จึงทำให้เกิดความผิดพลาดได้

Hint: ให้ดูการทำงานในระดับ low level, สมมุติว่า ภาษา assembly ของ `count++` คือ

```
load reg1 &count // อ่านค่าจากตัวแปร count ใน memory มายัง reg2 register
add reg2 reg1 1 // reg2 = reg1 + 1
store &count reg2 // เขียนค่าจาก reg2 ไปเก็บในตัวแปร count ใน memory
```

ข้อ 4: จงค้นคว้าหาานิยามบน Internet และ AI ของ concepts ต่อไปนี้ว่าคืออะไร และนำมาเขียนตอบตามความเข้าใจของตนเองด้วยคำพูดของ นศ. เอง

1. Race condition
2. Thread-safe (ตัวอย่าง คนมักถามว่า function นี้จาก library นี้ thread-safe ไหม เช่น strtok() thread-safe ไหม หรือ libatopng thread-safe ไหม เป็นต้น thread-safe หมายความว่าอย่างไร และ function ที่ไม่ thread-safe คือ function ที่มีคุณสมบัติอย่างไร)
3. Critical section
4. Mutual exclusion

ข้อ 5: จงศึกษาโปรแกรม mutexex1.c ใน Slide 08 และ Slide 09 และขยายโปรแกรมให้ทำสิ่งต่อไปนี้

1. ให้โปรแกรมรับ argument ทาง command line หนึ่งค่า สมมุติว่าชื่อ N โดยกำหนดให้ N เป็นค่า positive integer และเมื่อ นศ. สั่งรันโปรแกรมด้วยคำสั่ง (สมมุติว่า executable โปรแกรมชื่อ mutexex1 ถ้าออกคำสั่ง)
 - a. `$ ./mutexex1 150`
 - b. ให้หมายถึง N = 150 และโปรแกรมสร้าง thread 150 เทรด (ไม่รวม main thread)
 - c. กำหนดให้เทรดแต่ละเทรดมีค่า Integer เป็นค่า rank_id จาก 0 ถึง N-1 (นศ. ต้องขยายโปรแกรมให้แต่ละ Thread รับค่า rank_id ที่ main thread กำหนดให้)
2. กำหนดให้โปรแกรมมี global variable เป็นตัวแปร integer ชื่อ global_var มีค่าเริ่มต้นเท่ากับ 25
3. หลังจาก main thread สร้าง thread N threads แล้ว ให้ main thread เพิ่มค่า global_var เป็นจำนวน 75 เหมือนในไฟล์ mutexex1.c
4. กำหนดให้ starting routine ของแต่ละ Thread ปฏิบัติตามเงื่อนไขต่อไปนี้
 - a. ถ้า rank_id เป็นเลขคู่ ให้เพิ่ม global_var อีก 25
 - b. ถ้า rank_id เป็นเลขคี่ ให้ลด global_var ลง 10

ข้อ 6: จงศึกษาการใช้งาน pthread_detach() จาก

- i. http://embeddedguruji.blogspot.com/2015/05/multithreading-in-c-tutorial-part_15.html และ
- ii. https://thispointer.com/posix-detached-vs-joinable-threads-pthread_join-pthread_detach-examples/

และเขียนโปรแกรมเพื่อสร้าง thread (ด้วยคำสั่ง `pthread_create()`) เพื่อเรียก function เพื่อพิมพ์ “Hello World” ตามด้วยค่า rank id ออกสู่หน้าจอจำนวน 50 threads และกำหนดให้ทุก thread มี id ตั้งแต่ 0 ถึง 49 และจงตอบคำถามต่อไปนี้

1. จงอธิบายว่า joinable thread คืออะไร เพราะเหตุใด เมื่อมีการใช้ `pthread_join()` เพื่อรอให้เทรดจบการประมวลผลเราจึงไม่จำเป็นต้องเรียก `pthread_detach()` เพื่อเคลียร์ข้อมูลเกี่ยวกับเทรดนั้น
2. อธิบายว่า `pthread_detach()` มีหน้าที่อะไร
3. โปรแกรมของ นศ พิมพ์ค่า “Hello World” ตามด้วยค่า rank id ออกสู่หน้าจอ ครบทุกเทรดหรือไม่ ถ้า main thread จบก่อนจะอย่างไร ถ้าใส่คำสั่ง `sleep` ใน main thread เพื่อรอจะป้องกัน race condition ได้อย่างถาวรหรือไม่ อะไรคือข้อเสียของการ detach
4. ถ้า main thread ของโปรแกรม ขึ้น prompt มาถามผู้ใช้หลังจากสร้างเทรดทั้งหมดแล้วว่า “do you want to terminate program (y/n)?” และให้ terminate program ถ้าผู้ใช้ตอบ “y” จะแก้ปัญหา main thread จบก่อนโดยที่ผู้ใช้ไม่ได้ตั้งใจได้หรือไม่ จงเพิ่ม code ในโปรแกรม

ข้อ 7: จงเขียนโปรแกรม เพื่อจำลอง `pthread_join()` โดยใช้ตัวแปร mutex ในภาพที่ 1 และภาพที่ 2 แล้วตอบคำถามต่อไปนี้

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 pthread_mutex_t mutex_var;
7
8 void * foo(void * arg){
9     pthread_t tid = pthread_self();
10    printf("foo tid = %lu do something\n", (unsigned long)tid);
11
12    sleep(5);
13
14    printf("foo unlock mutex\n");
15    pthread_mutex_unlock(&mutex_var);
16    pthread_exit((void *)NULL);
17 }
18
```

ภาพที่ 3

```

19 int main(void){
20
21     pthread_t ntid;
22     void *tret;
23
24     pthread_mutex_init(&mutex_var, NULL);
25
26     pthread_create(&ntid, NULL, foo, NULL);
27     pthread_detach(ntid);
28     printf("main thread do something \n");
29
30     //pthread_join(ntid, tret);
31     pthread_mutex_lock(&mutex_var);
32     printf("main thread wait here...\n");
33     pthread_mutex_lock(&mutex_var);
34     printf("main thread enters critical section and leave...\n");
35     pthread_mutex_unlock(&mutex_var);
36
37 }

```

ภาพที่ 4

1. จากภาพจงอธิบายพฤติกรรมการประมวลผลของโปรแกรม ในกรณีที่ เทรด main ประมวลผลบรรทัดที่ 31 และบรรทัดที่ 33 ก่อนที่ เทรด foo จะประมวลผลคำสั่งในบรรทัดที่ 15
2. จากภาพจงอธิบายพฤติกรรมการประมวลผลของโปรแกรม ในกรณีที่ เทรด main ประมวลผลบรรทัดที่ 31 แล้วหลังจากนั้นเทรด foo ประมวลผลคำสั่งในบรรทัดที่ 15 และสุดท้ายเทรด main ประมวลผลคำสั่งในบรรทัดที่ 33
3. จากภาพจงอธิบายพฤติกรรมการประมวลผลของโปรแกรม ในกรณีที่ เทรด foo จะประมวลผลคำสั่งในบรรทัดที่ 15 ก่อนที่ เทรด main จะประมวลผลบรรทัดที่ 31 และบรรทัดที่ 33 จงอธิบายว่าเกิดปัญหาอะไรขึ้นในกรณีนี้
4. จงปรับปรุงโปรแกรมให้ถูกต้องโดยย้ายคำสั่ง lock ในบรรทัดที่ 31 ไปไว้ก่อนการสร้างเทรด foo ในบรรทัดที่ 26 และอธิบายเพราะเหตุใดการเปลี่ยนแปลงดังกล่าวจึงสามารถแก้ไขปัญหในข้อ c ได้
5. จงหาข้อมูลจาก Internet และ AI ว่าโปรแกรมนี้ไม่ตรงกับ POSIX standard อย่างไร ให้อ้างอิงแหล่งที่มาด้วย

ข้อ 8: จงเขียนโปรแกรมจากภาพที่ 5 และ 6 เพื่อสร้างสถานการณ์ deadlock ของสองเธรด

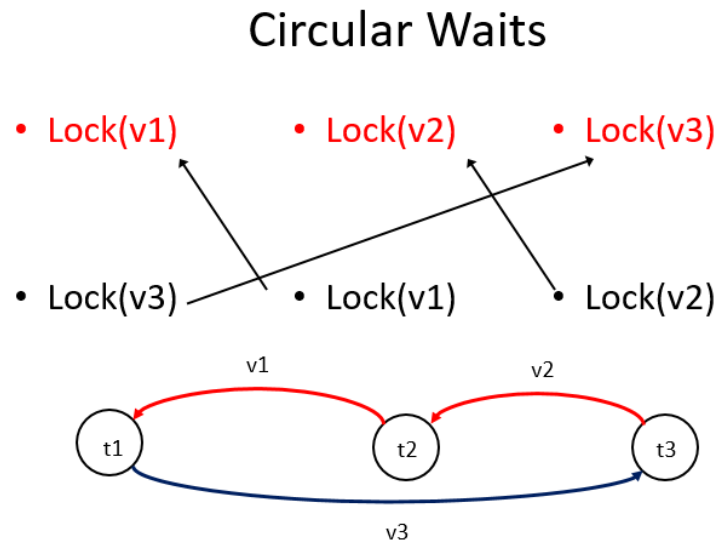
```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 pthread_mutex_t v1, v2;
7
8 void * t(void * arg){
9     long int id = (long) arg;
10    if(id == 1){
11        pthread_mutex_lock(&v1);
12        sleep(2);
13        pthread_mutex_lock(&v2);
14        printf("enter critical section of t(1)\n");
15        pthread_mutex_unlock(&v2);
16        pthread_mutex_unlock(&v1);
17    }
18    else if(id == 2){
19        pthread_mutex_lock(&v2);
20        sleep(2);
21        pthread_mutex_lock(&v1);
22        printf("enter critical section of t(2)\n");
23        pthread_mutex_unlock(&v1);
24        pthread_mutex_unlock(&v2);
25    }
26    pthread_exit((void *)NULL);
27 }
```

ภาพที่ 5

```
29 int main(void){
30
31     pthread_t ntid1, ntid2;
32
33     pthread_mutex_init(&v1, NULL);
34     pthread_mutex_init(&v2, NULL);
35
36     pthread_create(&ntid1, NULL, t, (void *)1);
37     pthread_create(&ntid2, NULL, t, (void *)2);
38
39     pthread_join(ntid1, NULL);
40     pthread_join(ntid2, NULL);
41
42 }
```

ภาพที่ 6

ข้อ 9: จงขยายจากโปรแกรมในข้อก่อนหน้าเพื่อสร้างโปรแกรมที่ทำให้เกิด deadlock ที่สร้าง circular wait แบบ 3 เทรดดังภาพที่ 7



ภาพที่ 7

ข้อ 10: จงเขียนโปรแกรมเพื่อสร้าง threads ขึ้น 5 เทรด ให้มีค่า rank id ตั้งแต่ 1 ถึง 5 โดยให้แต่ละเทรดได้แก่ thread rank 1 ถึง rank 5 คำนวณค่าในแต่ละ equation (1) ถึง (5) ข้างล่าง ตามอันดับ

$$a = b + c + d + e \quad (1)$$

$$b = a + c + d + e \quad (2)$$

$$c = a + b + d + e \quad (3)$$

$$d = 100 \quad (4)$$

$$e = a + d \quad (5)$$

โดยมีเงื่อนไขต่อไปนี้

1. ค่าเริ่มต้นของตัวแปรทุกตัวก่อนโปรแกรมปฏิบัติงานตาม equation ข้างต้นคือ 1
2. สมมุติว่าทีมโปรแกรมเมอร์ที่ออกแบบโปรแกรมนี้นึกถึงกันว่าจะสร้างตัวแปร Mutex ดังตารางที่ 1 ได้แก่ mutex v_1 ถึง v_5 เพื่อใช้เป็นตัวแปรกำกับการเปลี่ยนแปลงค่าแบบ mutual exclusion ของ variable a ถึง e ที่จะถูกเรียกใช้ในเทรด ตามอันดับ

ตารางที่ 1

ตัวแปร MUTEX	กำกับการเข้าถึง (อ่านและเขียน) ค่าตัวแปรใน EQUATION
V1	a
V2	b
V3	c
V4	d
V5	e

จงเขียนโปรแกรม multi-threads ตามข้อกำหนดข้างต้น โดยใช้หลักการ Deadlock Prevention ที่ศึกษาใน slide 08 และ อธิบายว่าเพราะเหตุใดโปรแกรมนี้จึงไม่เกิด deadlock